Implementation of Power Efficient Posit Multiplier

VLSI Architecture Course Presentation

Department of ECE

30/04/26

Posit Format & Numerical Analysis

Posit Components: (Nb, es)

- **Sign (s):** 0 for positive, 1 for negative.
- **Regime (rg):** Variable length prefix.
- **Exponent (exp):** es bits of scaling.
- **Fraction ($frac$):** Mantissa bits.

$$value = (-1)^s \times useed^k \times 2^{exp} \times (1 + frac)$$

$$useed = 2^{2^{es}}$$

Example: -255.03125 **($Nb = 16, es = 2$)**

- **Binary:** $-1.111111100001_2 \times 2^7$
- **Fields:** $k = 1$, $exp = 3$ (11_2), $s = 1$.
- **Assembly:** 1 110 11 1111111000
- **2's Comp:** 1 001 00 0000001000

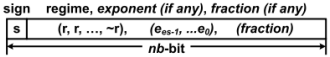


Fig. 1. Each component of a posit number.

Step-by-Step Calculation:

- $useed = 2^{2^2} = 16$
- $2^7 = 16^1 \times 2^3 \implies k = 1, exp = 3$

Regime Encoding Mapping (k):

binary	001×	01××	10××	110×	1110
value of k	-2	-1	0	1	2

Input Data Extraction Hardware

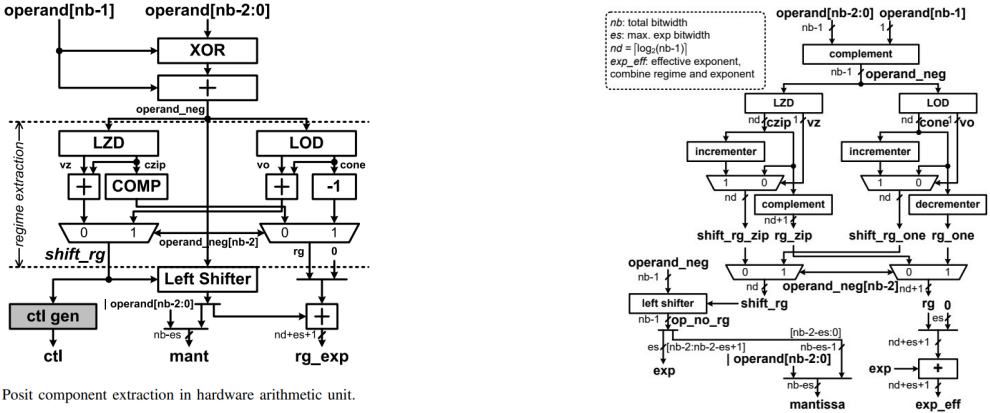


Fig. 3. Posit component extraction in hardware arithmetic unit.

Source: Thesis on "Flexible Multiple-Precision Fused Arithmetic Units for Efficient Deep Learning Computation" by Hao Zhang

Posit Multiplier Structure

- Standard flow:
  1. Input extraction
  2. Mantissa multiplication
  3. Add regime/exponent
  4. Normalize
  5. Pack output posit
- Mantissa multiplier generally designed for **max width**:

$$(nb - es)\text{-bit}$$

- Causes unnecessary switching when actual mantissa is small
- Leads to higher dynamic power

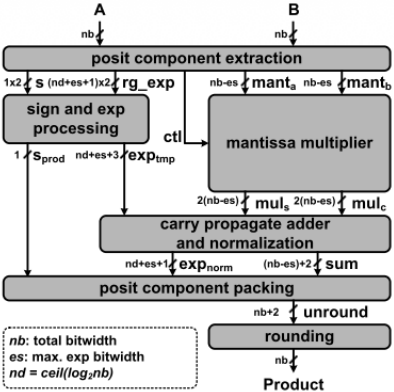


Fig. 4. Datapath of the proposed posit multiplier.

Proposed Paper Novelty

Key Idea: Divide multiplier into smaller regions

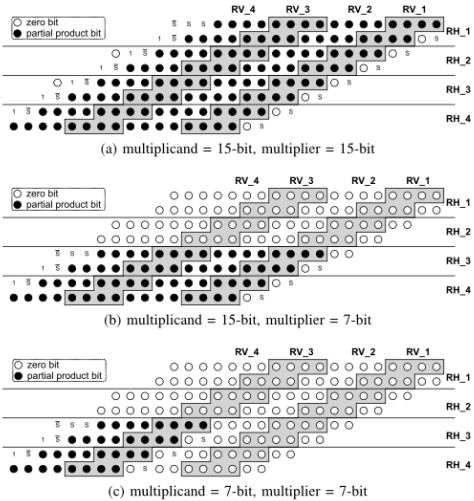
Only required regions are enabled at runtime.

Control generated from regime width:

$$mant_bit = nb - es - shift_{rg}$$

$$ctl[1] = \overline{shift}_{rg}[3], \quad ctl[0] = \overline{shift}_{rg}[2]$$

- Lower switching activity
- Lower power consumption
- Small timing/area overhead



(a) multiplicand = 15-bit, multiplier = 15-bit

(b) multiplicand = 15-bit, multiplier = 7-bit

(c) multiplicand = 7-bit, multiplier = 7-bit

Fig. 5. The proposed mantissa multiplier decomposed into multiple regions.

Proposed Novelty

TABLE I
TRUTH TABLE TO GENERATE CONTROL SIGNAL

shift_rg	*ctl*	Enable				*shift_rg*	*ctl*	Enable			
		R_1	R_2	R_3	R_4			R_1	R_2	R_3	R_4
0000	xx					1000	01			✓	✓
0001	xx					1001	01			✓	✓
0010	11	✓	✓	✓	✓	1010	01			✓	✓
0011	11	✓	✓	✓	✓	1011	01			✓	✓
0100	10		✓	✓	✓	1100	00				✓
0101	10		✓	✓	✓	1101	00				✓
0110	10		✓	✓	✓	1110	00				✓
0111	10		✓	✓	✓	1111	00				✓

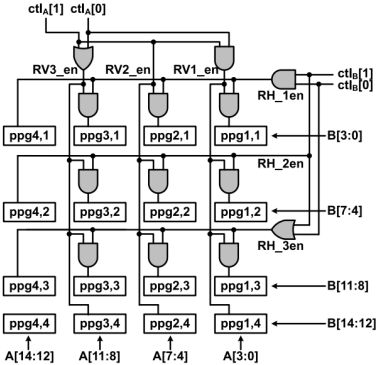


Fig. 6. The use of *ctl* signal as the enable signal for partial product generation (ppg) in mantissa multiplier (Posit(16,1) example). ppg(x,y) refers to the ppg used to generate partial products in region RV_x and RH_y.

Our Implementation: Mantissa Multiply Replacement

What We Changed

- Used standard **PACoGen** as baseline
- Replaced direct mantissa multiply block
- Added custom **4-region radix-4** multiplier
- Same output signal: `mult_m`

Benefits

- Lower switching activity
- Reduced dynamic power
- No outer datapath changes

Code Replacement

Original (PACoGen)

```
wire [2*(N-es)+1:0] mult_m = m1*m2;
```

$\Downarrow$

Replaced With Our Module

```
mantissa_mult_4r4 #(.MANT_W(MANT_W)) u_mm (
  .A(m1), .B(m2),
  .ctl_A(ctl_A_2b), .ctl_B(ctl_B_2b),
  .product(mult_m)
);
```

Inside Our 4-Region Multiplier (N=16)

Mantissa split into four 4-bit regions

$$A = [A_4\ A_3\ A_2\ A_1], \quad B = [B_4\ B_3\ B_2\ B_1]$$

	B_4	B_3	B_2	B_1
A_4	×	×	×	×
A_3	×	×	×	×
A_2	×	×	×	×
A_1	×	×	×	×

Green = enabled PPG blocks Gray = gated OFF blocks

- Each cell is one gated `4-bit` × `4-bit` multiplier
- Only required regions are active based on `ctl_A`, `ctl_B`
- Active partial products are shifted and summed to generate `mult_m`

Adaptation for N=8 and N=32 Formats

For N=8

Used `mantissa_mult_2r`

- Mantissa split into two 4-bit regions
- Only 4 partial blocks
- 1-bit control:

$$ctl_A,\ ctl_B$$

- Lower hardware overhead for small design

$$2 \times 2 = 4 \text{ blocks}$$

For N=32

Used `mantissa_mult_4r8`

- Mantissa split into four 8-bit regions
- Each cell uses ppg8
- Total 16 region products
- 2-bit control retained

$$4 \times 4 = 16 \text{ blocks}$$

Reason:
Using 8-bit cells reduces area and wiring compared to many 4-bit cells.

Hence architecture scales with posit size while preserving same top-level datapath.

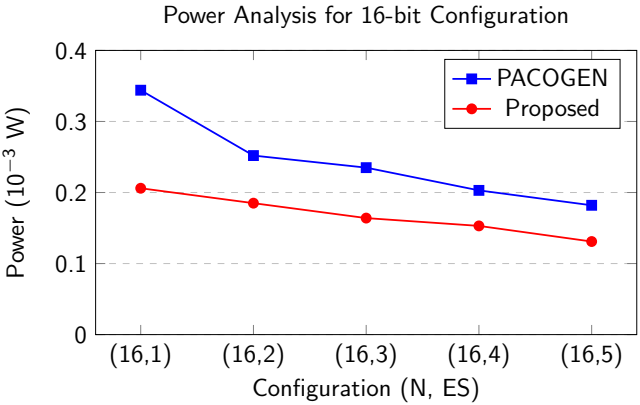
Synthesis Results (45nm GPDK)

Design	Delay(ns)	Area(μm^2)	Power(mW)	Power Red.(%)
(8,1)-PACoGen	7.863	728.46	0.0635	–
(8,1)-Proposed	8.007	759.58	0.0552	13%
(16,1)-PACoGen	15.108	2236	0.344	–
(16,1)-Proposed	14.831	2668	0.206	40%
(16,2)-PACoGen	14.606	2084	0.252	–
(16,2)-Proposed	14.812	2410	0.185	26.5%
(32,1)-PACoGen	25.978	6799	1.33	–
(32,1)-Proposed	25.573	8957	0.703	47.1%
(32,2)-PACoGen	25.829	6592	1.24	–
(32,2)-Proposed	25.498	8560	0.680	45.1%

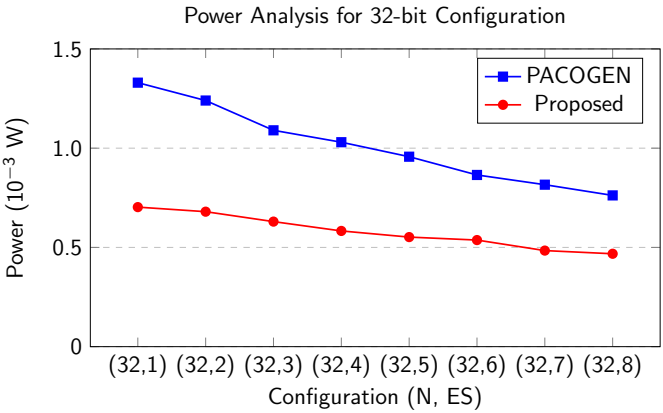
**Values rounded for readability. Minor variation across synthesis runs is expected.*

**(8,0) not evaluated since PACoGen does not support es=0.*

Power Comparison for $N = 16$



Power Comparison for $N = 32$



Thank You